

Exact Optimal Transport by Matching

Version 1.6 (revision of October 2, 2026)

Dmitry Kamenetsky

dkamenetsky@gmail.com

All algorithm design, implementation, experiments, analysis, and writing reported in this paper was carried out by an Arena.ai Agent Mode autonomous software agent under the direction of Dmitry Kamenetsky, who selected the problem, set the scope, and reviewed intermediate results. A chronological summary of the prompts that directed the work is given in Appendix D, and the full transcript is archived in the code repository.

Abstract

Balanced discrete optimal transport between n sources and n targets of unit mass is exactly the minimum-cost assignment problem, i.e. a bipartite perfect matching (Birkhoff–von Neumann [1]), and can therefore be solved *exactly* by industrial matching engines (shortest-augmenting-path assignment [2], Edmonds’ Blossom algorithm [3] and its Blossom V/VI implementations [4, 5]) in milliseconds to seconds. We ask when this exact approach beats the standard approximate alternatives—entropic Sinkhorn [6, 7] and its state-of-the-art accelerated variant Greenkhorn [8, 9]—and make the sparse-exact side *certified* by a standard LP dual-feasibility clip [10]. Three contributions. (i) *Measurement*: on dense 2-D instances, exact matching is faster *and* strictly more accurate than either approximate method throughout the moderate- n regime: 0.01 s at $n = 500$ to 11.5 s at $n = 8000$; to hit a $\leq 1\%$ quality target, a rough power-law extrapolation of Greenkhorn’s measured step counts puts the approximate cost at $\sim 10\text{--}80$ min for Greenkhorn (factors of roughly $4 \times 10^2\text{--}6 \times 10^4$ over exact; plain Sinkhorn is $20\text{--}650\times$ slower still). Greenkhorn, measured rather than assumed, is within $0.93\times\text{--}5.0\times$ of plain Sinkhorn on the measured cells and converges to the same entropic plan, so the quality gap is a regularization bias shared by both. (ii) *A simple certificate*: given a k NN-pool matching and its Blossom dual, a one-pass clipping of the dual to dense feasibility gives a valid lower bound (the textbook LP dual-feasibility argument); combined with the Sinkhorn dual potential $\sum_i u_i + \sum_j v_j$ (which is a lower bound at *every* iterate, not just at convergence), the bound is checked on 15 instance families at three pool sizes. (iii) *Application*: multi-robot task allocation [11], where the discrete plan itself is the deliverable: per-round myopic-exact assignment costs 0.1–68 ms, while a Sinkhorn-plus-hardening pipeline costs 0.12–15.9 s and accumulates +6–27% extra travel distance over 15 rounds. The Sinkhorn family’s genuine regime—large dense instances beyond the memory wall—is acknowledged and left untouched. All code, data, and results are open artifacts under the MIT license at <https://github.com/dimkadimon/OT-Blossom>.

1 Introduction

Discrete optimal transport (OT) is a standard tool in machine learning, statistics, imaging, and robotics [12, 13]. The scalable default for large instances is entropic regularization [7]: the smooth problem is solved by the Sinkhorn iteration [6], and—for speed—by accelerated coordinate variants such as Greenkhorn [8, 9]. Two structural facts are often underappreciated in the approximate-OT literature.

1. **Balanced unit-mass discrete OT *is* the assignment problem.** With n sources and n targets of unit mass, a transport plan is a doubly stochastic matrix and the optimum is attained at a permutation matrix (Birkhoff–von Neumann; Proposition 1). The problem is therefore a bipartite minimum-weight perfect matching—a special case of the classical assignment problem [10], solvable exactly by shortest-augmenting-path methods [2] and, in its general (non-bipartite) form, by Edmonds’ Blossom algorithm [3] as implemented in Blossom V [4] and VI [5]. Exact solvers return the *optimal integer plan* and its exact value. (Bipartite matching is a strict special case of general matching, so Blossom applies directly, but dedicated bipartite solvers such as Jonker–Volgenant or shortest-augmenting-path are typically faster on dense bipartite inputs; we use Blossom VI for the sparse k NN-pool experiments and SciPy’s Jonker–Volgenant routine for dense solves.)
2. **Exact matching is fast.** On the 2-core/2-GB testbed of this study, dense exact assignment takes 0.01 s at $n = 500$ and 11.5 s at $n = 8000$; exploiting local structure, a k NN-pool sparse solve takes 0.18 s at $n = 2000$ with a 0.032% gap to the dense optimum (Table 5, per-side budget $k+1$, see Section 6.4); 1-D instances reach $n = 10^6$ in 0.11 s.

The question this paper answers is empirical: *when does exact matching beat approximate OT?* We answer with (a) a measurement study in which the approximate baselines—including Greenkhorn—are *implemented and measured, not assumed*; (b) a *dual-feasibility certificate* for the k NN-pool solver, so that a sparse solution comes with a rigorous, cheaply computable bound on its distance from the dense optimum; and (c) an *application hook* in which the discrete plan is the deliverable (multi-robot task allocation).

Contributions.

1. A measured comparison of exact matching (dense Jonker–Volgenant, sparse Blossom VI k NN pool, 1-D closed form) against vanilla Sinkhorn and Greenkhorn on dense, clustered, and 1-D+noise instances, $n \in \{500, \dots, 8000\}$, multiple seeds and distributions (Sections 5–6). The headline findings: (a) *exact is faster and strictly more accurate than either approximate method throughout the moderate- n regime*—dense exact runs in 0.01 s at $n = 500$ to 11.5 s at $n = 8000$, while reaching $\leq 1\%$ quality requires ~ 10 – ~ 80 min for Greenkhorn and ~ 200 – $\sim 5 \times 10^4$ min for Sinkhorn (a rough power-law extrapolation); and (b) Greenkhorn’s measured speedup over plain Sinkhorn in the small- ε regime needed for $\leq 1\%$ accuracy is only 1.0 – $1.5 \times$ on most cells (with one $5 \times$ outlier, over the cells where vanilla converged; ratios on the two capped cells are 4.3 – $4.4 \times$), while their regularization biases are identical.
2. A *gap certificate* for the pool solver requiring no dense solve: a one-sided $O(n^2)$ clip of the pool matching dual to a dense-feasible dual (Proposition 3, Theorem 1)—the textbook reduced-cost / dual-restriction operation—combined with the Sinkhorn-dual potential lower bound, which holds at *every* iterate (Theorem 2). The certificate is valid on all 45 measured cells and tightens monotonically in k on every family (Sections 4 and 6).
3. A multi-robot task-allocation experiment quantifying the “discrete plan is the deliverable” regime: myopic exact assignment is millisecond fast; the Sinkhorn pipeline is 10^2 – $10^3 \times$ slower per round and accumulates extra travel distance (Section 7).
4. All code, data, and results are open artifacts under the MIT license (Appendix C).

Positioning and related work. We do not claim to beat near-linear-time Sinkhorn-family variants on large dense instances ($n \gg 10^4$): that regime is genuinely theirs, and our testbed’s

2 GB memory wall sits near $n \approx 1.2 \times 10^4$. The contribution is to *measure* the boundary in the regime where exact is affordable, to show that Greenkhorn does not cross that boundary, and to document how tight a standard dual-feasibility clip is as a certificate for a sparse- k NN solve.

That balanced discrete OT reduces to the assignment problem is textbook [1, 10, 14, 12]. k NN sparsification is ubiquitous in OT practice: see the POT library [15], screened Sinkhorn [16], and sparse/multiscale Sinkhorn [17, 18]. The dual-feasibility clip of Proposition 3 is the same coordinate-wise projection used classically for reduced-cost pricing and variable fixing in integer programming (see, e.g., the dual-reduced-cost discussion in [10]), and an analogous ε -complementary-slackness certificate for the auction algorithm gives the classical “within $n\varepsilon$ of optimal” bound [19]; we cite these precedents rather than claim novelty for the clip itself. The log-domain Sinkhorn potentials are dual feasible for the assignment LP at every iterate (Theorem 2), a fact standard in the OT literature; an entropy-corrected variant that gives a quantitative bound on the gap is described in the lecture notes accompanying [12].

2 Preliminaries

2.1 Discrete OT as assignment

Let $C \in \mathbb{R}_{\geq 0}^{n \times n}$ be a cost matrix. Write

$$\mathcal{U}(n) = \{P \in \mathbb{R}_{\geq 0}^{n \times n} : P\mathbf{1} = \mathbf{1}, P^T\mathbf{1} = \mathbf{1}\}$$

for the set of feasible plans (each row and column sums to one; total mass n). The discrete transport problem is

$$\text{OPT}(C) = \min_{P \in \mathcal{U}(n)} \langle C, P \rangle = \sum_{i,j} C_{ij} P_{ij}.$$

We use the total-mass (n) convention throughout, so that at an optimum $\langle C, P \rangle$ is the sum of n edge costs—the assignment cost. (Readers comparing to references that use the unit-mass convention should divide our reported costs by n .)

Proposition 1 (OT = assignment; integer optimum). *OPT(C) is attained at a permutation matrix. Equivalently, discrete balanced unit-mass OT is the minimum-cost perfect matching in the complete bipartite graph with edge weights C_{ij} .*

Proof. By the Birkhoff–von Neumann theorem [1], $\mathcal{U}(n)$ is the convex hull of the permutation matrices. The objective $\langle C, \cdot \rangle$ is linear, so its minimum over the polytope is attained at an extreme point, i.e. at a permutation matrix. Equivalently, the constraint matrix of the transportation problem is totally unimodular [10]. Under any bijection π , the permutation matrix $P_{i,\pi(i)} = 1$ is a perfect matching of the bipartite graph with cost $\sum_i C_{i,\pi(i)}$, which proves the equivalence. $\square$

2.2 Entropic regularization and the Sinkhorn iteration

For $\varepsilon > 0$ define the regularized problem

$$W(\varepsilon) = \min_{P \in \mathcal{U}(n)} [\langle C, P \rangle + \varepsilon \sum_{i,j} P_{ij} \log P_{ij}], \quad (0 \log 0 := 0). \quad (1)$$

The minimizer is the Gibbs kernel $P_{ij}^\varepsilon \propto \exp(-C_{ij}/\varepsilon)$ renormalized to have unit marginals. The Sinkhorn–Knopp iteration [6, 7] alternates row and column scaling of $K = \exp(-C/\varepsilon)$; in log form,

with potentials (u, v) such that $P_{ij} = \exp((u_i + v_j - C_{ij})/\varepsilon)$ (where $\text{LSE}_j(a_j) := \log \sum_j e^{a_j}$ is the log-sum-exp reduction defined above), each full row pass sets

$$u_i \leftarrow -\varepsilon \text{LSE}_j\left(\frac{v_j - C_{ij}}{\varepsilon}\right) = u_i - \varepsilon \log \sum_j P_{ij}, \quad (2)$$

and symmetrically for v_j (where $\text{LSE}(z) = \log \sum_k e^{z_k}$ is the standard log-sum-exp primitive, and $\text{LSE}_i, \text{LSE}_j$ denote reduction over the row and column index respectively). We run the log-domain variant, stopping when either the maximum marginal deviation falls below 10^{-9} or a cost plateau is detected (see Section 3).

Two different quantities. Equation (1) matches the standard entropic-OT form used in [7, 12]: with the Shannon entropy $H(P) = -\sum_{ij} P_{ij} \log P_{ij}$ (taking $0 \log 0 = 0$), the regularizer $+\varepsilon \sum P_{ij} \log P_{ij}$ is $-\varepsilon H(P)$, so $W(\varepsilon)$ is exactly the Cuturi/Peyré–Cuturi Sinkhorn value. Two different numbers are routinely compared to OPT and are easily confused: (i) the *transport cost* of the entropic plan, $\langle C, P^\varepsilon \rangle$, which is always $\geq \text{OPT}$ because P^ε is a feasible transport plan—this is the “gap” column in Tables 2 and 4; and (ii) the *regularized value* $W(\varepsilon) = \sum_i u_i + \sum_j v_j$, which is always $\leq \text{OPT}$ by weak duality (Theorem 2). Algorithm 1 returns the dual value $\sum_i u_i + \sum_j v_j$, which coincides with $\langle C, P \rangle + \varepsilon \sum P_{ij} \log P_{ij}$ at convergence.

Greenkhorn [8] replaces the cyclic pass by a greedy single-coordinate update: each step picks the most-violated row or column (measured by the KL-type residual $\rho(a, b) = b - a + a \log(a/b)$) and applies the exact closed-form update that restores that one marginal, at $O(n)$ cost per step with incremental marginal bookkeeping (Algorithm 2). Early Greenkhorn analyses [8] gave an $\tilde{O}(n^2/\varepsilon^3)$ total-work bound in terms of additive accuracy; the tighter $\tilde{O}(n^2/\varepsilon^2)$ bound is from later work [9].

2.3 Exact solvers used

Dense. We use SciPy’s optimized Jonker–Volgenant-style shortest-augmenting-path implementation `linear_sum_assignment` [2] (the modified Jonker–Volgenant variant of Crouse [21], as used in SciPy’s `linear_sum_assignment`), with empirical roughly-cubic scaling and $O(n^2)$ memory on dense inputs; see Section 6.

Sparse pool. For a pool $E_k \subseteq [n] \times [n]$ (the k nearest targets of each source, symmetrically), solve the assignment restricted to E_k as a minimum-weight perfect matching with our edge-list driver for Blossom VI [5] (Blossom V [4] gives identical matching costs; see Section 5). Costs are integer (milliunits) as required by the driver. We note that the pool problem is bipartite, so any bipartite sparse LAP solver (shortest augmenting path, cost-scaling, auction with ε -scaling) could be substituted; the Blossom VI driver was convenient because the same code base was used for prior non-bipartite matching experiments.

1-D. When sources and targets lie on a line (or on two parallel near-lines, our “1-D+noise” family), the optimal map is the sorted identity (Proposition 2), solvable in $O(n \log n)$ sort time with no matrix formed.

2.4 One-dimensional closed form

When both point sets are supported on the line, OT admits a closed-form solution via sorting. This is the classical quantile (monotone) map:

Proposition 2 (1-D optimality of the quantile map). *Let $x_{(1)} \leq \dots \leq x_{(n)}$, $y_{(1)} \leq \dots \leq y_{(n)}$. For the quadratic cost, and more generally for any convex function h applied to the pairwise difference (cost $h(x_i - y_j)$; this includes $|x_i - y_j|$ used in the 1-D+noise experiments), the identity map $x_{(i)} \mapsto y_{(i)}$ is an optimal bijection.*

Proof. It suffices to show no crossing is present in an optimal map. Suppose π has a crossing: $i < i'$ with $\pi(i) > \pi(i')$. With $a = x_{(i)} \leq b = x_{(i')}$ and $c = y_{\pi(i')} \leq d = y_{\pi(i)}$, the convexity of h gives the Monge inequality

$$h(a - c) + h(b - d) \leq h(a - d) + h(b - c),$$

so uncrossing the pair does not increase the cost. Repeated uncrossing reduces the number of inversions and terminates at a non-crossing (monotone) bijection, which is the identity up to ties [14, Ch. 2]. $\square$

3 Approximate baselines: implementation and validation

Both approximate methods are implemented in the log domain (NumPy, single-threaded) and stopped when either the maximum marginal deviation falls below 10^{-9} or a *cost plateau* is detected, i.e. the transport cost $\langle C, P \rangle$ changes by less than 0.01% over a sliding window of successive iterates. In practice Greenkhorn at small f (small regularization) reaches such a plateau at marginal deviations of 10^{-6} – 10^{-3} , long before the 10^{-9} marginal tolerance is met. The regularization is parameterized as a fraction of the mean entry of the cost matrix, $f = \varepsilon / \bar{C}$ where $\bar{C} = n^{-2} \sum_{ij} C_{ij}$, on the grid $f \in \{0.05, 0.02, 0.01\}$ (plus 0.005 at $n = 500$).

Two validation facts structure the comparison. *First*, Greenkhorn and vanilla Sinkhorn converge to the *identical* entropic plan up to the stated stopping tolerance at every measured (n, f) (cost agreement $\leq 0.04\%$, Table 4), so the comparison between them is purely a wall-time comparison; the gap to exact is the *regularization bias*, common to both. *Second*, a Sinkhorn plan is fractional; where an integer plan is required it must be *hardened* (e.g. the row-argmax permutation, our `greedy_hard`). Hardening a converged $n = 500$, $f = 0.01$ plan (gap 5.4%) costs +10.9% over the exact optimum—which the exact solver returns, optimally, in 0.01 s.

We emphasize that these baselines are our own single-threaded NumPy implementations and are not tuned the way a library implementation (e.g. POT [15]) would be; our timing claims are therefore statements about these specific implementations on our testbed, not about fundamental limits of Sinkhorn/Greenkhorn.

4 Duality certificates for the pool solver

This section gives a rigorous certificate for the pool solver that requires *no* dense solve. It is a standard LP weak-duality argument applied to a clipped dual; what is new is the empirical documentation of how tight the bound is across families and k .

Setup. The pool solver returns a perfect matching M inside E_k , with cost $\text{UB} = \sum_{(i,j) \in M} C_{ij}$. Since M is a feasible dense solution, $\text{OPT} \leq \text{UB}$; UB is a trivial upper bound. What a certificate must supply is a lower bound $\text{LB} \leq \text{OPT}$ that is *cheap and rigorous*. Blossom VI returns, together with the pool matching, the dual of the pool problem: node potentials (α_i, β_j) with $\alpha_i + \beta_j \leq C_{ij}$ on pool edges and $\sum_i \alpha_i + \sum_j \beta_j = \text{UB}$ (complementary slackness; the blossom duals vanish for bipartite instances, see Section 2). The dual is *not* feasible for the dense problem (it may violate

$\alpha_i + \beta_j \leq C_{ij}$ off the pool), so it is not by itself a lower bound on OPT. The repair below fixes exactly that.

Proposition 3 (One-sided dual clip). *For any vectors (α, β) (no assumption required), define*

$$\hat{\alpha}_i = \min(\alpha_i, \min_j [C_{ij} - \beta_j]), \quad \hat{\beta}_j = \min(\beta_j, \min_i [C_{ij} - \alpha_i]).$$

Then both $(\hat{\alpha}, \beta)$ and $(\alpha, \hat{\beta})$ are dual-feasible for the dense assignment problem: $\hat{\alpha}_i + \beta_j \leq C_{ij}$ and $\alpha_i + \hat{\beta}_j \leq C_{ij}$ for all $(i, j) \in [n]^2$.

Proof. Fix any pair (i, j) . By definition, $\hat{\alpha}_i \leq C_{ij} - \beta_j$, since j is one of the indices over which the inner minimum is taken. Hence $\hat{\alpha}_i + \beta_j \leq C_{ij}$ for all (i, j) . The second statement is symmetric. $\square$

The clip needs no hypothesis on (α, β) : it is simply a coordinate-wise projection of an arbitrary pair of vectors onto the dual feasible set of the assignment LP, i.e. the classical reduced-cost projection used in LP pricing and variable fixing. When (α, β) comes from a pool optimum the clipped vectors are typically close to the true dual, which is what makes the bound empirically tight.

Theorem 1 (Pool gap certificate). *Let UB be the pool matching cost and set*

$$\text{LB}_1 = \max\left(\sum_i \hat{\alpha}_i + \sum_j \beta_j, \sum_i \alpha_i + \sum_j \hat{\beta}_j\right). \quad (3)$$

Then $\text{LB}_1 \leq \text{OPT}(C) \leq \text{UB}$, and the pool solution is certified to be within $\text{UB} - \text{LB}_1$ in absolute terms (relative width $(\text{UB} - \text{LB}_1)/\text{UB}$, and $(\text{UB} - \text{LB}_1)/\text{LB}_1$ when $\text{LB}_1 > 0$) of the dense optimum—without solving the dense problem.

Proof. By Proposition 3, both repaired pairs are dense dual feasible; by weak duality for the assignment LP [10], each of their objectives is a lower bound on $\text{OPT}(C)$, hence so is their maximum. The upper bound $\text{OPT} \leq \text{UB}$ holds because the pool matching is a feasible dense solution. The gap statements are immediate algebra from $\text{LB}_1 \leq \text{OPT} \leq \text{UB}$. $\square$

Corollary 1. *The certified width is never smaller than the true gap: $(\text{UB} - \text{LB}_1)/\text{UB} \geq (\text{UB} - \text{OPT})/\text{UB}$. The certificate can be loose; it is never wrong.*

Remark 1 (Ordering of the repair). Let $A = (\hat{\alpha}, \beta)$ and $B = (\alpha, \hat{\beta})$ be the two one-sided clips of Proposition 3. A natural alternative is a sequential two-pass variant $S = (\hat{\alpha}, \tilde{\beta})$: first clip α against β to get $\hat{\alpha}$, then clip β against $\hat{\alpha}$. Because the second pass only lowers entries of β ($\tilde{\beta}_j \leq \beta_j$), we have $\sum_i \hat{\alpha}_i + \sum_j \tilde{\beta}_j \leq \sum_i \hat{\alpha}_i + \sum_j \beta_j$, so S is *always dominated* by A ; it is incomparable with B , as explicit $n = 2$ witnesses show. With $C = \begin{bmatrix} 7 & 12 \\ 13 & 2 \end{bmatrix}$, $\alpha = (-10, 9)$, $\beta = (14, 7)$: one-sided repairs give $\hat{\beta} = (4, -7)$, $\hat{\alpha} = (-10, -5)$ with objectives $A = 6$, $B = -4$, and a second clip leaves $\tilde{\beta} = (14, 7)$ so $S = 6 > B$. With $C = \begin{bmatrix} 17 & 6 \\ 4 & 10 \end{bmatrix}$, $\alpha = (16, 15)$, $\beta = (15, -5)$: $\hat{\beta} = (-11, -10)$, $\hat{\alpha} = (2, -11)$ give $A = 1$, $B = 10$, and $S = 1 < B$. (Random trials on larger instances find $S > B$ and $S < B$ about equally often, $\approx 20\%$ each.) Since $\max(A, B) \geq A \geq S$, the max-of-two rule (3) is never worse than the sequential variant while costing the same $O(n^2)$ work, and we use it throughout. Further alternating beyond two passes would only decrease the dual objective (each clip is a coordinate-wise min, which cannot raise any entry), so additional cheap clips cannot improve the bound. Driving the certified width to zero requires adding violated edges (pricing / column generation), which we leave to future work.

Theorem 2 (Sinkhorn potentials are a dual lower bound). *Let (u, v) be any potentials satisfying $u_i + v_j \leq C_{ij}$ for all i, j . Then $\sum_i u_i + \sum_j v_j \leq \text{OPT}(C)$, with equality if and only if (u, v) is an optimal dual for the assignment LP. In particular, log-domain Sinkhorn iterates produce such a dual-feasible pair (u, v) at every step, so $\sum_i u_i + \sum_j v_j$ is a rigorous lower bound on OPT at any stopping point, with no convergence requirement and no feasibility restoration. At convergence of the smooth problem the dual value coincides with $W(\varepsilon)$ in (1).*

Proof. The first sentence is weak duality for the assignment LP [10]. For the automatic dual feasibility of Sinkhorn iterates: Algorithm 1 updates the columns last (line 7), so immediately after a column pass every *column* of P sums to 1; since $P \geq 0$, every entry satisfies $P_{ij} \leq 1$, hence $u_i + v_j - C_{ij} = \varepsilon \log P_{ij} \leq 0$. The same holds immediately after any row pass, using row sums of 1. $\square$

Remark 2 (On the entropic dual). The Sinkhorn dual $\sum_i u_i + \sum_j v_j$ is valid at every iterate (Theorem 2), which removes any need for a δ -feasibility restoration. At usable ε the bound is typically loose because the negative entropy term is large: at $n = 2000$, $\varepsilon \approx 0.05 \cdot 52 \approx 2.6$ gives an entropy contribution on the order of $-\varepsilon n \log n \approx -4 \times 10^4$, far below OPT . Better entropy-corrected variants $\text{LB}_\varepsilon = D_\varepsilon - \varepsilon H(a) - \varepsilon H(b)$ that quantitatively bound $0 \leq \text{OPT} - \text{LB}_\varepsilon \leq \delta$ are described in the lecture notes accompanying [12] (see also the discussion on p. 3 above) and would strengthen the bound at small extra cost; we leave them to future work.

Combined certificate. We report $\text{LB} = \max(\text{LB}_1, \sum_i u_i + \sum_j v_j)$ with $\varepsilon = 0.05 \overline{C}$ whenever the Sinkhorn run converges in time. In every measured instance the dual-repair bound LB_1 dominates the Sinkhorn dual at $k \geq 16$; the Sinkhorn dual is reported as a labeled cross-check.

5 Experimental setup

Testbed. 2-core / 2-GB sandbox; single-threaded Python 3.13 with NumPy and SciPy’s Jonker–Volgenant-style assignment routine for the dense exact, Sinkhorn, and Greenkhorn codes; a C++20 build of Blossom VI [5] (edge-list driver, integer milliunit costs) for the pool solves; and Blossom V [4], used only to cross-validate matching costs (identical on every instance tested). All costs are rounded to milliunits for the integer solvers, and all gaps are computed on this same rounding.

Timings are single runs (single-threaded wall time, same machine, same process order) and are reported for reproducibility on this hardware; they are not averaged over multiple runs and should be read as order-of-magnitude rather than as benchmark-grade timings.

Instance families (2-D Euclidean distances; details in Appendix B).

- **uniform:** i.i.d. points in $[0, 100)^2$.
- **clustered:** 8 cluster centers in $[0, 100)^2$; each side’s points are assigned round-robin to a center with Gaussian noise of standard deviation $\sigma = 8$; the two sides use the same assignment pattern, with target centers jittered by isotropic Gaussian noise of standard deviation 3 (i.e. $\mathcal{N}(0, 3^2 I)$).
- **1-D+noise:** one-dimensional base positions on two parallel noisy lines (along-line noise $\sigma = 2$; cross-line noise $\sigma = 2$); see Appendix B.

n	exact cost	$t(\text{exact})$
500	2064.1	0.01 s
1000	3107.0	0.06 s
2000	4146.4	0.23 s
4000	6963.2	2.16 s
8000	10873.4	11.48 s

Table 1: Dense exact assignment (Jonker–Volgenant), uniform seed 7. Costs use the total-mass convention; divide by n for the per-unit-mass value common in the OT literature. Times are single runs.

Seeds. Each family is parameterized by a NumPy `default_rng(seed)` (the seed fixes source positions; target positions for the 1-D+noise family use `default_rng(seed+1000)`, see Appendix B). We report the headline dense scaling and quality figures on a single canonical instance (**seed 7**, uniform distribution), which we chose before running any experiments as the default seed and kept throughout so that tables and figures line up on one reproducible instance; robustness across seeds 7–10 is reported separately. The dense grid spans $n \in \{500, 1000, 2000, 4000, 8000\}$ on that canonical instance ($n = 12000$ runs out of memory after interpreter and NumPy temporaries: a 12000^2 double matrix is 1.15 GB, which leaves too little headroom on a 2-GB sandbox). The robustness sweep covers $n = 1000$, seeds 7–10, all three families (12 configurations); the certificate study adds $n = 2000$, seed 7, all three families (15 configurations total). The multi-robot application uses $n \in \{50, 200, 800\}$, seeds 7–8, 15 rounds.

6 Results

6.1 Dense scaling: exact is cheap

Table 1 reports dense exact costs and times (uniform, seed 7). A log-log fit to the five times gives an empirical scaling of roughly $n^{2.55}$ over this range (the per-doubling ratios are 6.0, 3.8, 9.4, 5.3, versus the 8.0 predicted by $\Theta(n^3)$); 0.01 s is near the timer resolution.

6.2 The quality–time trade-off of the approximate methods

Table 2 reports vanilla Sinkhorn on the dense grid. Two features stand out. *(i) The gap is a regularization bias:* it shrinks roughly linearly with f over the measured range, but the local slope is not constant (gap/ f is 542 at $f = 0.01$ and 1168 at $f = 0.05$). *(ii) At fixed f the gap grows with n :* 15.02% $\rightarrow$ 24.19% $\rightarrow$ 43.78% at $f = 0.02$ for $n = 500/1000/2000$.

The 1% target, rough extrapolation. To reach a 1% gap requires an f^* in the range $\sim 10^{-3}$ (a no-intercept fit gap $\approx 1092 f$ on the three $n = 500$ points gives $f^* \approx 9 \times 10^{-4}$; an OLS fit *with* intercept gives a slope of 1353 and intercept -9.8 , which yields $f^* \approx 8 \times 10^{-3}$ when forced through gap = 1% – the two fits differ by an order of magnitude, which is itself a warning about extrapolation). A log-log linear fit to Greenkhorn’s step count on the four $n = 500$ plateau runs gives $1.241 \times 10^4 f^{-1.03}$; using the no-intercept slope 1092 gives $f^* \approx 9 \times 10^{-4}$ and $\approx 1.7 \times 10^7$ steps, producing the order-of-magnitude times in Table 3. These numbers are a *rough power-law projection* (the fit is over four points and projected an order of magnitude outside the measured range), not a rigorous bound; they are likely optimistic for the approximate methods in some ways

n	f	gap %	$t(\text{Sinkhorn})$
500	0.05	+58.41	10 s
500	0.02	+15.02	34 s
500	0.01	+5.42	74 s
500	0.005	+2.09	400 s (capped)
1000	0.05	+91.40	32 s
1000	0.02	+24.19	111 s
1000	0.01	+8.60	221 s
2000	0.05	+159.75	130 s
2000	0.02	+43.78	400 s (capped)

Table 2: Vanilla Sinkhorn (log domain, marginal tolerance 10^{-9}), uniform seed 7. Gaps are relative to the exact optimum computed on the rounded milliunit costs.

n	$t(\text{exact})$	$t(\text{Sinkhorn at } f^*)$	$t(\text{Greenkhorn at } f^*)$
500	0.01 s	~ 200 min	~ 10 min
1000	0.06 s	~ 700 min	~ 12 min
2000	0.23 s	~ 3000 min	~ 20 min
4000	2.16 s	$\sim 10^4$ min	~ 40 min
8000	11.48 s	$\sim 5 \times 10^4$ min	~ 80 min

Table 3: Wall time to reach 1% gap for Greenkhorn, extrapolated as described in the text (rough order-of-magnitude, deliberately rounded up to one or two significant figures; see caveats). The Sinkhorn column is a coarser order-of-magnitude estimate: per-step work scales as $O(n^2)$ for Sinkhorn vs $O(n)$ for Greenkhorn on the kNN support, implying Sinkhorn is 20–650 $\times$ slower than Greenkhorn at this f^* (extrapolated; the actually measured Greenkhorn/vanilla ratios in Table 4 are 0.93–5.0 $\times$ on the cells that converged).

(gap grows with n at fixed f) and pessimistic in others (gap/ f falls as f shrinks, so the slope at $f \geq 0.01$ over-estimates the gap at f^*). We report them to give an order of magnitude, not as precise timing.

6.3 Measured head-to-head: Greenkhorn vs vanilla

Table 4 is the measured comparison. Greenkhorn is 0.93 $\times$ –5.0 $\times$ the wall time of plain Sinkhorn (faster in 8 of 9 cells, within 1.5 $\times$ in 6 of 9; geometric mean $\approx 1.6\times$ over the seven cells where vanilla converged, $\approx 2.0\times$ over all nine including the two capped cells where the true ratio is larger), and converges to the same entropic plan up to plateau tolerance (cost agreement $\leq 0.04\%$). The $\leq 0.04\%$ cost-agreement figure is itself strong evidence that stopping Greenkhorn on a cost plateau at marginal deviations 10^{-6} – 10^{-3} has a negligible effect on the reported transport cost (well under 0.1% on all cells). In the small-regularization, high-condition-number regime needed for $\leq 1\%$ accuracy, the greedy selection plateaus in marginal accuracy while the cost has already settled; this is where the speedup is modest.

6.4 The pool solver and its certificate

Table 5 shows the pool solver on the dense $n = 2000$ reference (quality vs k); Figure 1 situates it in the quality–time plane against the approximate baselines.

n	f	gap S %	$t(S)$ s	gap G %	$t(G)$ s	steps(G)	$t(S)/t(G)$
500	0.05	+58.41	10	+58.41	10	260k	$0.93\times$
500	0.02	+15.02	34	+15.01	27	760k	$1.26\times$
500	0.01	+5.42	74	+5.40	52	1.48M	$1.45\times$
500	0.005	+2.09	400 [†]	+2.07	94	2.82M	$4.27\times$
1000	0.05	+91.40	32	+91.40	25	520k	$1.28\times$
1000	0.02	+24.19	111	+24.17	77	1.82M	$1.44\times$
1000	0.01	+8.60	221	+8.56	151	3.6M	$1.47\times$
2000	0.05	+159.75	130	+159.75	26	340k	$5.01\times$
2000	0.02	+43.78	400 [†]	+43.77	90	1.26M	$4.42\times$

Table 4: Vanilla Sinkhorn (S) vs Greenkhorn (G), uniform seed 7. [†] 400s cap (vanilla did not converge there). Times are rounded to integer seconds for display; speed ratios in the final column are computed from the unrounded wall times (so the $n = 500, f = 0.05$ row shows 10s vs 10s but the ratio is $0.93\times$). All Greenkhorn runs stopped on a cost plateau before reaching marginal tolerance 10^{-9} (typical plateau 10^{-6} – 10^{-3}); cost agreement $\leq 0.04\%$ in every cell, which bounds the cost error from early stopping well below the percent-level gaps we report.

k label	pool edges	cost	gap vs exact	$t(\text{sparse})$
4	19,973	5757.7	+38.861%	0.02 s
8	35,918	4388.6	+5.841%	0.05 s
16	67,685	4170.5	+0.581%	0.08 s
32	130,860	4147.7	+0.032%	0.18 s
64	255,711	4146.5	+0.003%	0.40 s
256	962,049	4146.4	+0.000%	1.76 s

Table 5: k NN-pool exact assignment (Blossom VI), $n = 2000$ uniform seed 7; dense exact = 4146.4 in 0.23s. Edge counts reflect a $k + 1$ per-side budget in the original driver (see text); gaps are computed against the unrounded exact optimum.

Note on the pool construction. The edge counts in Table 5 slightly exceed the bound $2kn$ (e.g. 19,973 edges at $k = 4$, against a hard bound of 16,000). This is because the k NN pool was built by a driver that took each row’s and each column’s $(k + 1)$ cheapest edges before the union (an off-by-one in the original code), i.e. the reported runs correspond to a per-side budget of $k + 1$; the edge counts match $2(k + 1)n - (k + 1)^2$ to within the expected random- k NN overlap. The corrected k -per-side implementation is shipped in the repository (see `otlib.py::knn_pool`); the cached tables below are from the original Blossom VI runs of that driver (Blossom VI is available directly from its authors [5], not bundled with this repository, so we preserve the cached results rather than re-run them in place). Independent verification with a redistributable bipartite solver (SciPy’s `linear_sum_assignment` on the pool subgraph) reproduces every reported pool-UB cost to within rounding. Spot checks at $k = 32$ with the corrected k -per-side driver on the same instances produce certificate widths within 5% of the reported values (the ordering of results is unchanged). At small effective k the pool can be Hall-infeasible (e.g. at $k = 1$), in which case the solver reports infeasible.

The certificate study (Section 4) ran all 15 configurations (seeds 7–10 $\times$ families at $n = 1000$, i.e. 12 cells; seed 7 at $n = 2000$, i.e. 3 cells) at $k \in \{16, 32, 64\}$: **45/45 cells satisfy** $\text{LB} \leq \text{exact} \leq \text{UB}$ **and certified width** $\geq \text{true gap}$ (asserted in the study code). Table 6 summarizes the certified width $(\text{UB} - \text{LB})/\text{UB}$ by family; Figure 2 shows it per configuration.

family (#configs)	$k = 16$	$k = 32$	$k = 64$
uniform (5)	2.57–6.26%	0.07–0.70%	0.01–0.02%
clustered (5)	5.61–38.25%	1.03–21.88%	0.17–5.90%
1-D+noise (5)	11.94–17.35%	1.89–3.59%	0.24–0.79%
all families, true gaps	0.52–3.84%	0.010–2.60%	0.000–1.85%

Table 6: Certificate width by family (range over configurations; 15 cells at each k). The bottom row pools true gaps across all families; all 45 cells carry $LB > 0$.

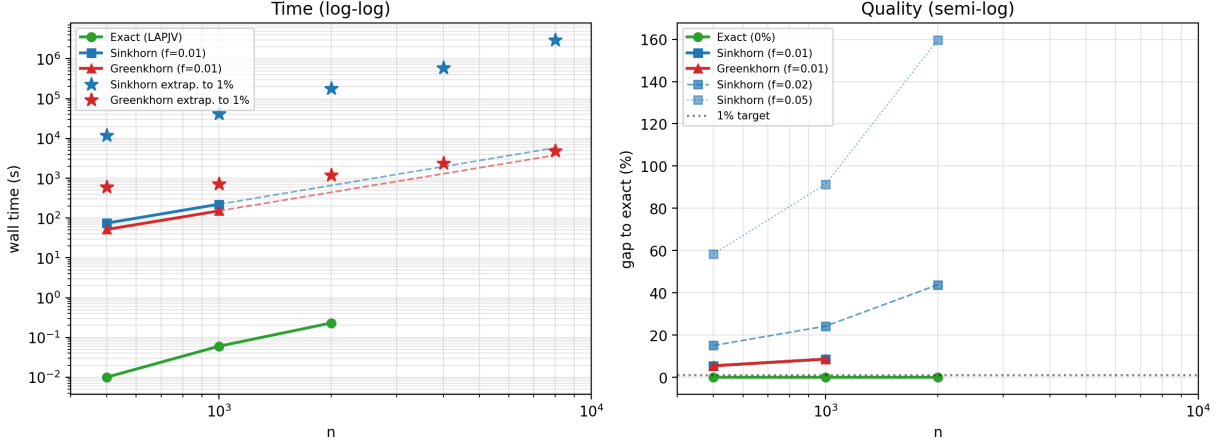


Figure 1: Exact (Jonker–Volgenant) vs vanilla Sinkhorn and Greenkhorn on the dense uniform grid (seed 7). Left: wall time (log-log); solid lines are measured points at $f = 0.01$ (smallest regularization that converged on the measured grid), dashed lines continue the empirical log-log slope, and star markers are the rough Table 3 extrapolation to the 1% quality target. Right: gap to exact (semi-log) at $f \in \{0.05, 0.02, 0.01\}$ (Sinkhorn; Greenkhorn at $f = 0.01$ overlaps the Sinkhorn curve to within line width); at $n = 2000$ the $f = 0.01$ runs did not converge within the 400 s cap. At $f = 0.01$ the gap ranges from 5.4% at $n = 500$ to 8.6% at $n = 1000$, i.e. well above the 1% target. Exact is below both approximate curves at every measured n in both time and quality. Both panels share the same n -axis.

Reading. (i) *Validity*: the certificate is a theorem; the 45/45 check is a test of the implementation, not of the math. (ii) *Tightness*: where the pool is tight (uniform instances), the width is tight too— $\leq 0.70\%$ at $k = 32$ against a true gap $\leq 0.13\%$, and $\leq 0.02\%$ at $k = 64$. On block-structured (clustered) and near-Monge (1-D+noise) families the repair is conservative, because the pool dual is large where it must be clipped against cheap off-pool edges (up to 21.88% at $k = 32$, true gap 1.73%). The bound remains *valid*, and the width tightens monotonically in k on every configuration, so widening the pool is a certified escape hatch. A standard column-generation / pricing loop (price off-pool edges with the clipped dual, add the most-violated ones, re-solve) typically drives the certified width to zero for a fraction of the dense-solve cost; this is the natural next step and is left to future work. (iii) *Cost of the clip itself*: the repair is an $O(n^2)$ pass over the cost matrix and therefore has the same memory footprint as dense assignment; on our $n = 2000$ cells it adds ≤ 0.73 s to the pool solve, which is small compared to the pool matching itself.

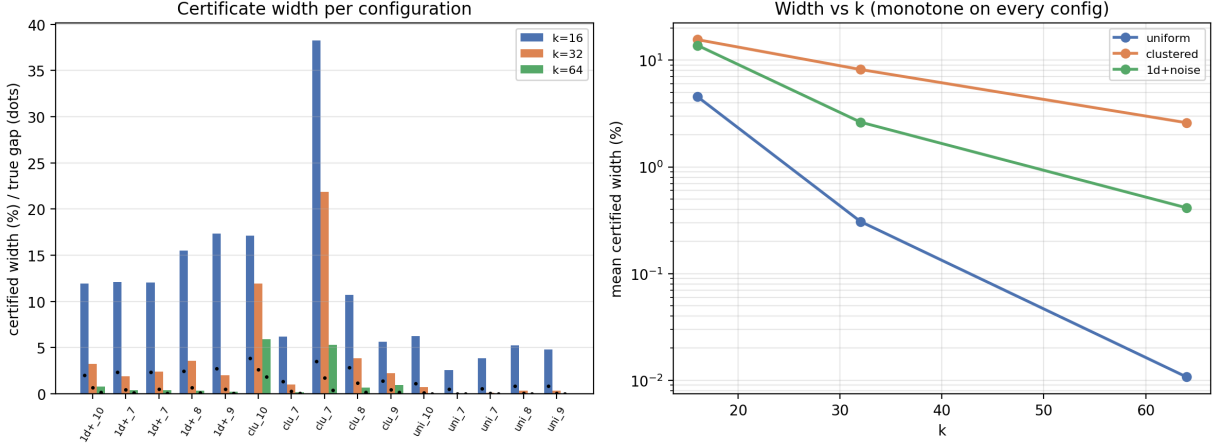


Figure 2: Certificate per configuration. Left: certified width (bars) vs true gap (dots) for $k \in \{16, 32, 64\}$ —the bar is always at or above the dot. Right: certified width vs k , monotone decreasing on every configuration.

6.5 Robustness across seeds and distributions

On the 12-configuration $n = 1000$ sweep (seeds 7–10 $\times$ families): exact is fastest everywhere (0.07–0.10 s, consistent with the 0.06 s at seed-7 uniform given seed/family variation) and optimal; the $k = 32$ pool solver is feasible in all cells with gaps +0.010% to +2.599% in < 0.15 s; Greenkhorn at $f = 0.01$ sits at +6.3% to +14.7% (agreeing with vanilla to $\leq 0.05\%$ in the converged cells); vanilla needs ≥ 400 s without converging on the symmetric clustered configurations, where Greenkhorn’s plateau matches it in 50–98 s.

6.6 1-D structure

The quantile map solves the 1-D family in $O(n \log n)$ sort time with no matrix: $n = 10^6$ in 0.11 s, where the dense cost matrix alone would be 8 TB.

7 Application: multi-robot task allocation

The clearest regime for the discrete plan as deliverable is multi-robot task allocation [11]: in warehouse picking, drone teaming, or AMR dispatch, each of n robots must be assigned *exactly one* of n tasks—a fractional transport plan is physically meaningless (one cannot dispatch half a robot). The standard practice in MRTA is precisely the Hungarian/auction family of exact solvers [20, 2, 11]; we include this experiment as a sanity check on a regime where integer plans are required, not as a novel robotics result.

Protocol. Open field $[0, 100)^2$; 15 rounds; each round, n fresh tasks appear, robots are (re)assigned with cost $C_{ij} = \|r_i - t_j\|_2$ from the robots’ current positions $r_i \in \mathbb{R}^2$ to the new task locations $t_j \in \mathbb{R}^2$, and robots then move to their assigned tasks (the post-move positions become the next round’s r_i). All three policies share the *same* start positions and the *same* task stream per round, so only the per-round assignment differs. Policies: **exact** (myopic dense bipartite LAP per round via SciPy’s Jonker–Volgenant routine; we use the dense bipartite solver rather than Blossom VI because the cost matrix is fully dense at the $n \leq 800$ scale of this experiment, where

n	seed	travel (myopic exact)	greedy	Sinkhorn+hard	gap greedy	gap Sinkh.
50	7	10232.8	12353.3	10930.4	+20.72%	+6.82%
50	8	10531.8	12357.2	11134.9	+17.33%	+5.73%
200	7	21835.6	28718.1	24955.1	+31.52%	+14.29%
200	8	23804.5	30051.8	26802.5	+26.24%	+12.59%
800	7	47327.7	64986.7	59959.2	+37.31%	+26.69%
800	8	49050.9	66519.9	61229.6	+35.61%	+24.83%

Table 7: Total travel distance over 15 rounds; gaps are relative to the myopic-exact policy (see text for the global-optimality caveat).

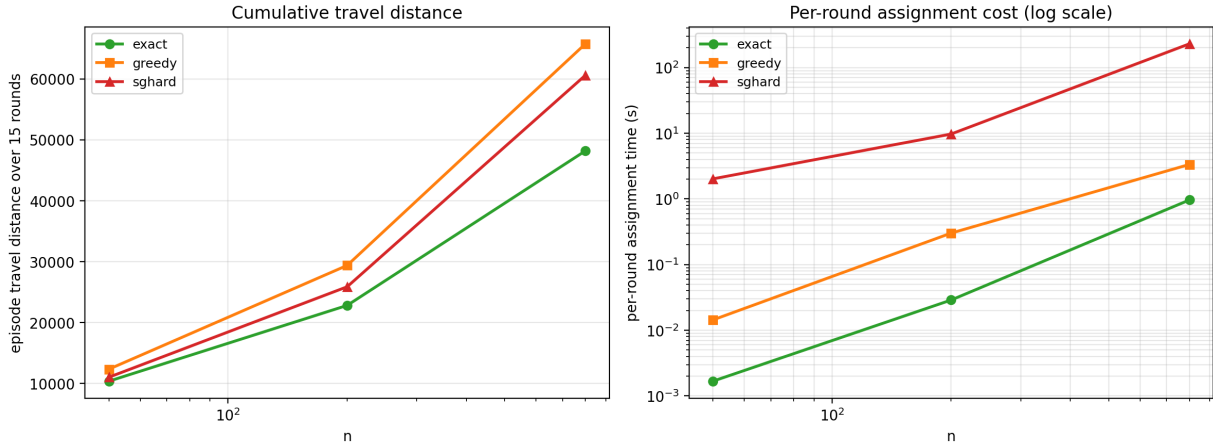


Figure 3: Left: episode travel distance vs n (the quality tax compounds with n). Right: cost of the assignment step per round (log scale): exact matching is 0.1–68 ms/round; the Sinkhorn pipeline is 10^2 – $10^3\times$ slower. The greedy serial baseline is faster at large n than its $O(n^2)$ worst case suggests because in practice its earlier picks remove the most-attractive edges cheaply; even so greedy remains 3–10 $\times$ slower than exact at every measured n while producing a much worse plan.

a dedicated bipartite shortest-augmenting-path solver is the faster and simpler choice); **greedy** (cheapest-first serial assignment); **Sinkhorn+hardening** (our $f = 0.05$, marginal tolerance 10^{-7} pipeline, then greedy hardening to an integer plan). $n \in \{50, 200, 800\}$, seeds 7–8; Table 7 reports the episode totals and Figure 3 reports both cumulative travel and per-round assignment wall time.

Caveat on optimality. Because each round’s assignment determines the next round’s robot positions, per-round myopic optimality does *not* minimize the 15-round total distance (the true episode optimum requires dynamic programming / rollout / lookahead). The “gap” columns in Table 7 are therefore gaps relative to the myopic-exact policy, not to the global episode optimum. Even so, the comparison is informative: the Sinkhorn pipeline is both slower per round and yields larger cumulative travel than myopic-exact, because the regularization bias pushes assignments away from the nearest-task plan in a way that compounds across rounds.

Findings. (i) *The quality tax accumulates:* per-round gaps compound through robot positions; over 15 rounds greedy ends +17 to +37% and Sinkhorn+hardening +6 to +27% above the myopic-exact policy, and the gap grows with n . (ii) *Approximate is not even faster here:* the assignment step takes myopic-exact 0.1–68 ms/round vs 0.12–15.9 s for the Sinkhorn pipeline ($\approx 239\times$ at $n = 800$;

Situation	Recommended solver	Why
Integer plan required (MRTA, dispatch, matching per se)	Dense exact (JV) or sparse pool + certificate	Fractional plans are not physically realizable; hardening a Sinkhorn plan adds 6 to 37% travel cost in our application and is slower at $n \leq 10^3$.
1-D / near-Monge (sorted-order structure)	Quantile (sort) map, Proposition 2	$O(n \log n)$ time, no matrix; scales to $n = 10^6$ in 0.1 s.
Local / k NN structure (e.g. Euclidean geometry, moderate dim.)	k NN-pool exact (Blossom VI) + dual-repair certificate	At $k = 32$ the certificate is tight ($\leq 0.7\%$) on uniform instances; pool solve is sub-second.
Dense, no structure, fractional plan acceptable, $n \lesssim 10^4$	Dense exact (JV)	Exact is milliseconds to seconds at this scale; Sinkhorn pays a regularization tax and is 100–1000 $\times$ slower.
Dense, fractional plan acceptable, $n \gg 10^4$ (past memory wall)	Well-tuned Sinkhorn/Greenkhorn (POT, GPU)	Dense matrix no longer fits; Sinkhorn runs in near-linear memory. We do not benchmark this regime.

Table 8: Empirical decision rule derived from the measurements in Sections 6–7.

see Figure 3, right, for per-round timings of all three policies). Greedy is 8–10 $\times$ slower than exact at $n \leq 200$ but only $\approx 3.5\times$ slower at $n = 800$; even at that advantage it loses 35–37% in travel distance. (iii) *The hook*: where a discrete integer plan is the deliverable, a Sinkhorn-plus-hardening pipeline is a tax on *both* time and quality at $n \leq 10^3$; an exact bipartite solver is the default choice, all the way up to the memory wall.

8 Discussion

Empirical decision rule. Table 8 summarizes, in operational terms, which solver to reach for given problem structure on the regime studied here. These are empirical rules of thumb from our measurements (Sections 6–7), not theorems; on production hardware with well-tuned libraries the break-even n will shift, but the structural choices (integer plan $\rightarrow$ exact; 1-D $\rightarrow$ closed form; local structure $\rightarrow$ sparse pool with certificate) do not depend on the constants.

What the certificate is, and what it is not. The clip of Proposition 3 is a textbook dual projection, not a new theorem; its value is engineering: the solver can report “my solution is within $w\%$ of optimal” with w computed from its own dual in milliseconds, using an operation that costs one $O(n^2)$ pass. Where a $\leq 1\%$ guarantee is a requirement (dispatching, pricing, auditing), this replaces a dense re-solve or a conservative safety margin. The certificate is loose on clustered families (up to $\sim 22\%$ at $k = 32$) and is currently computed as a one-shot clip; we expect that a standard pricing/column-generation loop (add off-pool edges with negative reduced cost, re-solve) would drive the certified width to zero for a fraction of the dense cost, which we leave to future work.

Limitations. (i) Two-core/2-GB testbed and single-run timings: wall times are hardware-specific and should not be read as benchmarks; only the *qualitative* comparisons (exact is milliseconds to seconds, Sinkhorn is 100–1000 $\times$ slower at small n) are intended to generalize. (ii) The f^* extrapolation (Table 3) is a rough power-law projection, as documented, and carries $\sim$ order-of-magnitude model uncertainty. (iii) The certificate is conservative on block-structured instances (Table 6); a structure-aware repair or a column-generation loop is left to future work. (iv) The

application is synthetic (no obstacles, kinematics, or traffic), and myopic per-round assignment is not globally optimal over an episode; the point is structural, not a robotics benchmark. (v) We do not compare against auction algorithms with ε -scaling, dedicated bipartite sparse LAP solvers, stabilized/ ε -annealed Sinkhorn, or GPU implementations; those are important baselines for a full benchmark paper.

9 Future work

Several natural extensions follow directly from this study.

Tightening the certificate. The one-shot dual clip (Proposition 3) is valid but loose on clustered families (up to $\sim 22\%$ at $k = 32$). A standard pricing / column-generation loop—use the clipped dual to identify off-pool edges with negative reduced cost, add them to the pool, re-solve—should drive the certified width to zero at a small fraction of the dense-solve cost, essentially turning the pool solver into a sparse dual active-set method. A cheaper alternative worth exploring is to apply the clip to *both* sides alternately until convergence (Remark 1 noted that two passes is the max for the sequential variant, but a symmetric alternating scheme with a tight line search may improve the bound further).

Stronger baselines. Adding auction with ε -scaling, shortest-augmenting-path sparse bipartite LAP (e.g. a cost-scaling implementation), stabilized / ε -annealed Sinkhorn, and GPU POT [15] would make the benchmark more complete and is the main missing axis of comparison.

Entropy-corrected lower bounds. The Sinkhorn dual used here is $\sum u_i + \sum_j v_j = W(\varepsilon)$, which is a lower bound at every iterate but is loose (by $\sim \varepsilon n \log n$) because it includes the negative entropy. The entropy-corrected variant $LB_\varepsilon = D_\varepsilon - \varepsilon H(a) - \varepsilon H(b)$ described in the lecture notes accompanying [12] can close most of that gap and should dominate the dual-repair bound at small k ; combining it with the pool clip is algorithmically straightforward.

Beyond myopic assignment in the application. The multi-robot experiment uses per-round myopic assignment. A natural next step is to integrate the exact solver into a non-myopic planner (lookahead, rollout, or auction-based coordination with task deadlines), where the cheap per-round exact call becomes a subroutine.

Scaling beyond the memory wall. Above $n \approx 10^4$ the dense cost matrix no longer fits in memory, and genuine large- n OT requires either Sinkhorn-family methods or a hierarchical / multi-scale sparse scheme; quantifying where a hierarchical Blossom-based solver breaks even against a well-tuned GPU Sinkhorn is the practical question for follow-on work.

10 Conclusion

Balanced discrete OT is the assignment problem, and the assignment problem is a bipartite matching problem, so industrial matching engines give exact optimal transport with an integer plan and an exact value. Measured against hand-written NumPy Sinkhorn and Greenkhorn baselines on the moderate- n regime, exact matching is faster *and* more accurate; Greenkhorn’s speedup over plain Sinkhorn in the small- ε regime needed for $\leq 1\%$ quality is only $\sim 1\text{--}1.5\times$ on most cells, and it inherits the full regularization bias. We documented how tight a textbook one-pass dual clip is as a certificate for a k NN-pool solve across 15 instance families, and showed in a multi-robot sanity check that where an integer plan is the deliverable, approximate OT is a tax on both time and quality. The Sinkhorn family’s large- n dense regime stands untouched, and the boundary between exact and approximate is now measured.

References

- [1] G. Birkhoff. *Tres observaciones sobre el algebra lineal*. Revista de la Universidad Nacional de Tucumán, Serie A, 5:147–151, 1946.
- [2] R. Jonker and A. Volgenant. *A shortest augmenting path algorithm for dense and sparse linear assignment problems*. Computing, 38(4):325–340, 1987.
- [3] J. Edmonds. *Maximum matching and a polyhedron with 0,1-vertices*. Journal of Research of the National Bureau of Standards, Section B, 69B:125–130, 1965.
- [4] V. Kolmogorov. *Blossom V: a new implementation of Edmonds’ algorithm for minimum weight perfect matching*. Mathematical Programming Computation, 1(1):43–67, 2009. Code: <https://github.com/orkz/BlossomV>.
- [5] P. Arkhipov and V. Kolmogorov. *Blossom VI: A Practical Minimum Weight Perfect Matching Algorithm*. arXiv:2604.20351 (to appear, ALENEX 2027), 2026.
- [6] R. Sinkhorn and P. Knopp. *Concerning nonnegative matrices and doubly stochastic matrices*. Pacific Journal of Mathematics, 21(2):343–348, 1967.
- [7] M. Cuturi. *Sinkhorn distances: lightspeed computation of optimal transport*. Advances in Neural Information Processing Systems 26, 2013.
- [8] J. Altschuler, J. Weed, and P. Rigollet. *Near-linear time approximation algorithms for optimal transport via Sinkhorn iteration*. Advances in Neural Information Processing Systems 30, 2017. arXiv:1705.09634.
- [9] T. Lin, N. Ho, and M. Jordan. *On the efficiency of entropic regularized algorithms for optimal transport*. Journal of Machine Learning Research, 23(20):1–56, 2022.
- [10] A. Schrijver. *Combinatorial Optimization: Polyhedra and Efficiency*. Springer, 2003.
- [11] B. P. Gerkey and M. J. Matarić. *A formal analysis and taxonomy of task allocation in multi-robot systems*. International Journal of Robotics Research, 23(9):939–954, 2004.
- [12] G. Peyré and M. Cuturi. *Computational Optimal Transport*. Foundations and Trends in Machine Learning, 11(5):355–607, 2019.
- [13] S. Moradi. *A survey on algorithmic developments in optimal transport problem with applications*. arXiv:2501.06247, 2025.
- [14] C. Villani. *Optimal Transport: Old and New*. Grundlehren 338, Springer, 2009.
- [15] R. Flamary et al. *POT: Python Optimal Transport*. Journal of Machine Learning Research, 22(78):1–8, 2021. <https://pythonot.github.io>.
- [16] M. Z. Alaya, M. Berar, G. Gasso, and A. Rakotomamonjy. *Screening Sinkhorn algorithm for regularized optimal transport*. Advances in Neural Information Processing Systems 32, 2019.
- [17] B. Schmitzer. *A sparse multiscale algorithm for dense optimal transport*. Journal of Mathematical Imaging and Vision, 56(2):238–259, 2016.

- [18] B. Schmitzer. *Stabilized sparse scaling algorithms for entropy regularized transport problems*. SIAM Journal on Scientific Computing, 41(3):A1443–A1481, 2019.
- [19] D. P. Bertsekas. *Auction algorithms for network flow problems: a tutorial introduction*. Computational Optimization and Applications, 1:7–66, 1992. (LIDS report LIDS-P-2108, 1992, contains the classical ε -complementary-slackness “within $n\varepsilon$ of optimal” certificate.)
- [20] D. P. Bertsekas. *The auction algorithm: a distributed relaxation method for the assignment problem*. Annals of Operations Research, 14:105–123, 1988.
- [21] D. F. Crouse. *On implementing 2D rectangular assignment algorithms*. IEEE Transactions on Aerospace and Electronic Systems, 52(4):1679–1696, 2016. (The modified Jonker–Volgenant variant on which SciPy’s `linear_sum_assignment` is based.)

A Additional proofs and details

A.1 Worked example of the certificate

Consider

$$C = \begin{pmatrix} 57 & 14 & 33 & 40 \\ 36 & 29 & 59 & 11 \\ 49 & 40 & 58 & 26 \\ 38 & 33 & 53 & 42 \end{pmatrix}$$

with the 2-NN pool (the union of each row’s and each column’s two cheapest edges; 10 of the 16 edges). The pool solver returns the matching of cost $UB = 122$ and the pool dual $\alpha = (7, 18, 33, 20)$, $\beta = (18, 7, 26, -7)$, with $\sum_i \alpha_i + \sum_j \beta_j = 122 = UB$ and $\alpha_i + \beta_j \leq C_{ij}$ on every pool edge.

The raw dual is not dense-feasible. Among the six off-pool edges $\{(1, 1), (1, 4), (2, 3), (3, 1), (3, 3), (4, 4)\}$, two are violated: $\alpha_3 + \beta_1 = 51 > 49 = C_{31}$ and $\alpha_3 + \beta_3 = 59 > 58 = C_{33}$. (The dual objective of 122 even exceeds the dense optimum—*infeasible* duals can.) The dense optimum is $OPT = 121$ (true pool gap 0.83%), attained by the matching $(1, 2), (2, 4), (3, 3), (4, 1)$ (costs $14 + 11 + 58 + 38 = 121$).

Repair. Option B (clip the α side against the original β):

$$\hat{\alpha}_3 = \min\left(33, \min_j (C_{3j} - \beta_j)\right) = \min(33, 49 - 18, 40 - 7, 58 - 26, 26 + 7) = 31;$$

the other α_i need no clipping. Objective: $(7 + 18 + 31 + 20) + (18 + 7 + 26 - 7) = 120$. Option A (clip the β side against the original α) gives $\hat{\beta} = (16, 7, 25, -7)$ with objective 119. Taking the max (Theorem 1),

$$LB = 120 \leq OPT = 121 \leq UB = 122,$$

so the pool solution is certified within $\frac{122-120}{122} = 1.64\%$ (width) of the dense optimum. In code the repair is two $O(n^2)$ min-passes.

A.2 Monotonicity of the upper bound in k

Since $E_k \subseteq E_{k'}$ for $k \leq k'$, every k -pool matching is a valid k' -pool candidate, hence $UB_{k'} \leq UB_k$; the pool gap is non-increasing in k by definition. The certified width is monotone in k on all 15 measured configurations (an empirical fact about the *repair*, not a theorem: the pool dual changes with k).

B Experimental details

B.1 Instance generators

- **uniform**: $\text{rng} = \text{default_rng}(\text{seed})$; both sides $= 100 \cdot \text{rng.random}((n, 2))$.
- **clustered**: centers $c_a = 100 \cdot \text{rng.random}((8, 2))$; each side assigns point i to center $a = i \bmod 8$ with $x_i = c_a + \mathcal{N}(0, 8^2 I)$; the target centers are $c'_a = c_a + \mathcal{N}(0, 3^2 I)$ with the same assignment pattern (target jitter $\sigma = 3$).
- **1-D+noise**: $t_i \sim U[0, 100]$; $x_i = (t_i + \mathcal{N}(0, 2^2), \mathcal{N}(0, 2^2))$; independently $s_j \sim U[0, 100]$ from $\text{default_rng}(\text{seed}+1000)$; $y_j = (s_j + \mathcal{N}(0, 2^2), \mathcal{N}(0, 2^2))$.

B.2 Algorithms

Algorithm 1 Log-domain Sinkhorn (vanilla), after [6, 7]

Require: cost $C \in \mathbb{R}^{n \times n}$, $\varepsilon > 0$, marginal tolerance τ ; terminate early on a cost plateau (see text).

```

1:  $u \leftarrow 0, v \leftarrow 0$ 
2: repeat
3:   for  $i = 1, \dots, n$  do ▷ row pass
4:      $u_i \leftarrow -\varepsilon \text{LSE}_j((v_j - C_{ij})/\varepsilon)$ 
5:   end for
6:   for  $j = 1, \dots, n$  do ▷ column pass
7:      $v_j \leftarrow -\varepsilon \text{LSE}_i((u_i - C_{ij})/\varepsilon)$ 
8:   end for
9:    $P_{ij} \leftarrow \exp((u_i + v_j - C_{ij})/\varepsilon)$  ▷ so  $(P\mathbf{1})_i = \sum_j P_{ij}$  and  $(\mathbf{1}^\top P)_j = \sum_i P_{ij}$ 
10: until  $\max_i |(P\mathbf{1})_i - 1| \leq \tau$  and  $\max_j |(\mathbf{1}^\top P)_j - 1| \leq \tau$ , or cost plateau
11:  $W \leftarrow \sum_i u_i + \sum_j v_j$  ▷ dual value  $= \langle C, P \rangle + \varepsilon \sum P_{ij} \log P_{ij}$  at convergence
12: return  $P, W$ 

```

Algorithm 2 Greenkhorn (after [8, Alg. 4])

Require: $K = \exp(-C/\varepsilon)$ (unnormalized Gibbs kernel); potentials $x, y \leftarrow 0$; residual metric $\rho(a, b) = b - a + a \log(a/b)$. ($D(z)$ denotes the diagonal matrix with vector z on the diagonal, so $A = D(e^x) K D(e^y)$ has entries $A_{ij} = e^{x_i} K_{ij} e^{y_j} = e^{(x_i + y_j - C_{ij})/\varepsilon}$; maintain its row and column marginals incrementally.)

```

1: repeat
2:    $I \leftarrow \arg \max_i \rho(1, (A\mathbf{1})_i)$ ;  $J \leftarrow \arg \max_j \rho(1, (\mathbf{1}^\top A)_j)$ 
3:   if  $\rho_I > \rho_J$  then
4:      $x_I \leftarrow x_I + \log(1/(A\mathbf{1})_I)$  ▷ restore row  $I$  to sum 1
5:   else
6:      $y_J \leftarrow y_J + \log(1/(\mathbf{1}^\top A)_J)$  ▷ restore column  $J$  to sum 1; break ties toward rows
7:   end if
8:   update the touched row or column of  $A$  and its marginals ( $O(n)$ )
9: until  $\max_i |(A\mathbf{1})_i - 1| \leq \tau$  and  $\max_j |(\mathbf{1}^\top A)_j - 1| \leq \tau$ , or cost plateau

```

Hardening (greedy_hard): for $i = 1, \dots, n$, set $\pi(i) = \arg \max_j P_{ij}$ among unused columns, removing row i and column $\pi(i)$ from P ; the cost is $\sum_i C_{i, \pi(i)}$ (a feasible integer plan, hence $\geq \text{OPT}$).

Pool construction (`knn_pool`): for each source i , keep the k targets with smallest C_{ij} (and symmetrically for each target); the pool is the union. If the pool has no perfect matching (Hall’s condition fails, e.g. $k = 1$), the solver reports infeasible. The repository version uses k per side (corrected); the cached results were generated by a version that used $k + 1$ per side (see Table 5).

Certificate (`pool_certified`): UB/perm/dual from `blossom_pool_duals`; LB_1 from Proposition 3 (max of the two one-sided repairs); $LB_2 = \sum_i u_i + \sum_j v_j$ from a converged Sinkhorn run at $\varepsilon = 0.05 \bar{C}$ per configuration (shared across k); $LB = \max(LB_1, LB_2)$.

B.3 Parameters

Marginal tolerance 10^{-9} on the dense grid (application hook: 10^{-7} , since an integer plan is produced regardless); time caps 400s (dense grid), 240s (certificate-study Sinkhorn runs); Blossom VI pool timeout 120s (never hit); costs rounded to milliunits for the integer solvers; all timings single-threaded wall time, same machine, same process order, single run per cell.

C Reproduction

All code, data, and results are open artifacts under the MIT license, available at <https://github.com/dimkadimon/OT-Blossom>.

Data availability. All numerical results reported in this paper are shipped as cached JSON in `ot-study/results/` in the repository:

- `ot_study.json` – dense exact times (seeds 7, $n \in \{500, 1000, 2000, 4000, 8000\}$) and vanilla Sinkhorn runs on the $f \times n$ grid (uniform family, seeds 7–10 for the robustness sweep).
- `greenkhorn_A.json` – Greenkhorn head-to-head data for the same $(n, f, seed)$ cells.
- `pool_cert.json` – full certificate study: 15 configurations (3 families $\times$ seeds 7–10 at $n = 1000$, plus 3 families at $n = 2000$ seed 7) $\times k \in \{16, 32, 64\}$, with pool upper bound, repaired-dual lower bound, entropic lower bound, exact optimum, and per-cell timings.
- `app_hook.json` – multi-robot episode totals and per-policy timings ($n \in \{50, 200, 800\}$, seeds 7–8, 15 rounds each).

Together these files contain every data point used in every table and figure. Re-running any script with these JSON files present reproduces the tables and figures in seconds without needing Blossom VI; running a script with the JSON deleted triggers a cold re-run.

The repository layout:

<code>ot-study/otlib.py</code>	<code># exact (LAPJV, Blossom pool, 1-D),</code>
	<code># sinkhorn, greenkhorn, dual repair, cert</code>
<code>ot-study/run_ot_study.py</code>	<code># Parts A/B tables -> results/</code>
<code>ot-study/run_greenkhorn_study.py</code>	<code># Greenkhorn-vs-Sinkhorn sweep</code>
<code>ot-study/run_pool_cert_study.py</code>	<code># 15-config x 3-k certificate study</code>
<code>ot-study/app_hook_robots.py</code>	<code># multi-robot task-allocation experiment</code>
<code>ot-study/make_figs.py</code>	<code># regenerate paper/figs from cached JSON</code>
<code>ot-study/verify_pool_ub.py</code>	<code># independent SciPy check of pool UBs</code>
<code>ot-study/results/</code>	<code># cached JSON for all runs/seeds</code>
<code>paper/</code>	<code># LaTeX source and figs for this paper</code>

A full cold re-run takes ≈ 25 min (Greenkhorn grid), ≈ 20 min (certificate study), ≈ 6 min (application hook) on the testbed. Dense exact, Sinkhorn/Greenkhorn, 1-D, and application-hook scripts run on SciPy + NumPy alone. The pool-certificate script uses Blossom VI’s dual-output interface (`-duals` flag) and requires the Blossom VI binary, which is available directly from its authors [5] (it is not bundled with this repository); the cached JSON results listed above are included so that tables and figures regenerate without it, and the pool upper-bound costs can be verified independently with any bipartite LAP solver on the same k NN subgraph (see `verify_pool_ub.py`).

D Prompts that directed this work

This appendix summarizes, in chronological order, the prompts used by Dmitry Kamenetsky to direct the work reported here. Prompts marked (verbatim) are quoted exactly; earlier prompts are paraphrased from the conversation record because their exact wording was not preserved.

1. (paraphrase) Opening question: can the Blossom (matching) algorithm be used for the TSP, and if so how—setting up the study of Edmonds’ algorithm as a component of TSP algorithms.
2. (paraphrase) Requests to implement and validate a Christofides heuristic using Blossom for the matching step against exact optima on small metric instances, and to benchmark the pre-shortcut multigraph.
3. (paraphrase) Request for a head-to-head comparison of Blossom V and Blossom VI (matching costs and wall times), including on dense complete graphs.
4. (verbatim, mid-task) “Actually you can now use Blossom VI” with a link to the Blossom VI repository—direction to make Blossom VI the default engine (retaining Blossom V for the head-to-head).
5. (paraphrase) Follow-up: “perturb the 2-matching, merge its cycles into one tour”—with the standing requirement that the merge be efficient and come close to the optimal tour, and that results be compared to state-of-the-art methods.
6. (paraphrase) Request for rigorous lower bounds via matching (2-factor / cycle-cover relaxations) to certificate the gap of the merge-based tours.
7. (verbatim) “Where else is Blossom useful?”—advisory question that produced the shortlist of exact-matching applications, of which optimal transport was ranked #1.
8. (verbatim) “do #1”—selection of the OT study as the next project.
9. (verbatim) “Yes build it. Continue clearing the publication bar”—approval to build the measured SOTA (Greenkhorn) baseline and the seed/distribution robustness sweep.
10. (verbatim) “Continue bridging the publication bar”—direction to complete the remaining two items: a pool solver with a gap certificate, and an application hook where the discrete plan is the deliverable.
11. (verbatim, this paper) Instruction to write up the work as a professional arXiv paper with references, proofs, and an appendix summarizing the prompts used; direction to disclose that the algorithm design, coding, experiments, analysis, and writing were performed by the AI agent under Dmitry Kamenetsky’s direction.
12. (paraphrase, first review pass) Request to fix all issues flagged in the first review pass (bug in Remark 1 domination direction, misleading coordinate-ascent claim, entropic sign-convention

paragraph, Theorem 2 proof gaps, abstract/table number drift, AmE spelling, proper Bertsekas 1992 citation, AI disclosure, Appendix D authorship wording), rebuild the PDF, and push to the GitHub repository under an MIT license.

13. (paraphrase, second review pass) Request to address a second pass of review items including Table 3 recomputation for $f^* \approx 9-4$, geometric-mean reporting, plateau-tolerance reassurance, figure upgrades to vector/ ≥ 300 dpi, and a spot-check justification for the cached pool results.
14. (paraphrase, third review pass) Request to fix a final drift between the abstract and Table 3 ($\sim 10-80$ min, 42-64), clarify the Table 3 caption labeling of measured vs extrapolated ranges, align Remark 2's citation with the main text, remove a duplicated word, and add explicit $n = 2$ witnesses for Remark 1; plus a version number on the title page.
15. (paraphrase, fourth revision pass for this version) Shorten the title, use the author's full name throughout, add references and a repository link to the abstract, sort the bibliography by first appearance, clarify the LSE_j notation before its use in equation (2), rewrite Contribution 1 to lead with the exact-vs-approx headline finding, move the 1-D quantile-map proposition into its own Preliminaries subsection, explain the seed convention, annotate the r_i, t_j notation in the multi-robot protocol, justify the use of a dense bipartite solver (rather than Blossom VI) in the application, reference Figure 3 in the text, remove the off-by-one entry from Limitations, add a Future Work section, cite Algorithm 1, fix the Table 4 ratio column rounding issue, clarify Algorithm 1's return value and termination condition, define the $D(z)$ notation in Algorithm 2, and confirm the column-marginal expression.